

Vamos a realizar una pequeña aplicación para gestionar la venta de entradas de los diferentes conciertos de una sala de espectáculos. El diseño de clases será el siguiente:

- Interfaz Entrada (entregada con el enunciado): los métodos de esta interfaz serán
 - boolean comprobarEntrada(int cod);
 - double devolverPrecioEntrada();
- Tendremos dos implementaciones posibles de la interfaz anterior: EntradaGeneral y EntradaVip, que se describen a continuación:
 - EntradaGeneral: se caracteriza por su código, su precio y el nombre del comprador
 - EntradaVIP: se caracteriza por su código, su precio, el nombre del comprador y la butaca asignada (una cadena de texto)

La implementación de los métodos de la interfaz Entrada para estas clases será el siguiente:

- *comprobarEntrada*: verificar que el código pasado por parámetro se corresponde con el de la entrada, si es así devuelve true y en caso contrario devuelve false
- *devolverPrecioEntrada*: devuelve el precio de la entrada

Puntuación: EntradaGeneral 0,5 pt. EntradaVIP 0,5 pt.

- Clase Concierto: se caracteriza por el nombre del grupo, precio de la entrada general, precio de la entrada VIP, conjunto de entradas vendidas, que será un Set de Entrada (a elegir implementación) y lista de entradas posibles, que será una lista de códigos (números enteros)

Puntuación: 0,5 pt.

- Nuestra aplicación además incorporará una gestión propia de excepciones de la siguiente manera: se declarará una clase excepción denominada ConciertosException (entregada con el enunciado), que será lanzada en las situaciones que se indican más adelante
- Clase Operaciones: contendrá las funcionalidades principales de la aplicación, que serán:
 - int venderEntrada(Concierto concierto): solicitará el tipo de entrada a vender, y posteriormente creará una entrada de dicho tipo con datos solicitados por teclado, salvo el código, que será el primer código de la lista de entradas posibles (que quedará eliminado de dicha lista) y el precio, que ya está definido en el objeto Concierto pasado por parámetro. Si la lista de códigos está vacía el método lanzará una excepción ConciertosException con un mensaje que explique la causa. Agregará esta entrada al conjunto de entradas vendidas para ese concierto si ha encontrado código

- `double ingresosConcierto(Concierto concierto)`: mostrará por consola todo lo que se ha ingresado con la venta de las entradas
- `Concierto conciertoMasExito(Concierto[] conciertos)`: dado un array de conciertos, devolverá como valor de retorno el concierto para el que más entradas se hayan vendido
- `void comprobarEntrada(Concierto concierto, int codigo)` : usando el método `comprobarEntrada` implementado por las clases, comprobará el código pasado por parámetro para todas y cada una de las entradas del concierto, en caso de que coincida con alguna de las entradas, mostrará los datos de la entrada (considera implementar *toString* en las clases `EntradaGeneral` y `EntradaVIP` para facilitarte esta tarea)
- `void comprobarTipoEntrada(String nombre, Concierto concierto)`: para el concierto pasado por parámetro, mostrará el tipo de entrada (VIP o General) del comprador cuyo nombre se ha pasado también, por parámetro. En caso de no encontrar a dicho comprador lanzará una excepción `ConciertosException` con un mensaje que explique la causa.

Puntuación:

- Cada uno de los cinco métodos anteriores puntuará **1,5 pt.**
- Cuestiones relacionadas con el diseño de las clases, utilización de los modificadores de acceso, polimorfismo (sobreescritura de métodos) **1 pt.**